RL MID-2 QUESTION BANK

DESCRIPTIVE QUESTIONS

UNIT-3

1. Explain the Markov property and its importance in MDPs.
2. Explain the agent–environment interaction in a finite MDP with a diagram.
3. Derive the unified notation for episodic and continuing tasks.
4. Explain the Bellman expectation equation for the state-value function with a Suitable example.
5. Derive the Bellman equation for the action-value function
6. Explain optimal value functions and their significance.

UNIT-4

1. Explain Monte Carlo prediction for estimating state-value functions.
2. Explain off-policy Monte Carlo control using importance sampling and discuss its advantages and limitations.
3. Describe the Monte Carlo Control with Exploring Starts algorithm with a suitable example.
4. Compare first-visit and every-visit Monte Carlo methods and explain their application in control problems.
5. Explain incremental implementation of Monte Carlo methods.

UNIT-5

1. Describe Samuel's Checkers Player and its learning mechanism.
2. Discuss the application of Reinforcement Learning in elevator dispatching.
3. Analyse how RL can be applied to solve Dynamic Channel Allocation problem based on network conditions.
4. Describe job-shop scheduling and the role of RL in optimizing schedules.

Note: For Mid-2, there will be no MCQ bit bank. Instead, the exam will have only 5 questions, each carrying 2 marks. A separate PDF containing the 2-mark questions with answers has been shared.

SYLLABUS(ACETR23):

UNIT-III:

Finite Markov Decision Processes: The Agent–Environment Interface, Goals and Rewards,Returns, Unified Notation for Episodic and Continuing Tasks, The Markov Property, Markov Decision Processes, Value Functions, Optimal Value Functions, Optimality and Approximation.

UNIT-IV:

Monte Carlo Methods: Monte Carlo Prediction, Monte Carlo Estimation of Action Values, Monte Carlo Control, Monte Carlo Control without Exploring Starts, Off- policy Prediction via Importance Sampling, Incremental Implementation, Off-Policy Monte Carlo Control, Importance Sampling on Truncated Returns

UNIT-V:

Applications and Case Studies: TD-Gammon, Samuel‘s Checkers Player, The Acrobot,

Elevator Dispatching, Dynamic Channel Allocation, Job-Shop Scheduling.

BHAAI PDFS

Reinforcement Learning (RL)

Unit III — Finite Markov Decision Processes

1. Define the terms Goal, Rewards and Returns.
2. Differentiate episodic and continuing tasks with examples and derive the unified notation for them.
3. Compare state-value and action-value functions.
4. Define Optimistic Initial Values.

Unit IV — Monte Carlo Methods

5. Define Monte Carlo Prediction.
6. Differentiate between exploration and exploitation.
7. Why is importance sampling used with truncated returns?
8. Differentiate between ordinary and weighted importance sampling.

Unit V — Applications and Case Studies

9. What is job-shop scheduling?
10. How can RL be applied in a gaming environment?



BHAAI PDFs

BY STUDENTS. FOR STUDENTS.

Keep the PDFs coming, bhaai.

Help us continue through 4-1 semester exams.

FUNDRAISING GOAL

₹400

Contribute ₹10 or more

Our ChatGPT Go subscription ends on **4 November 2026.**

Your support helps us renew it and keep preparing and sharing study PDFs.

A little support. More study resources.

LINK TO DONATE

https://bhaai-pdfs-support.vercel.app/

Thank you for supporting Bhaai PDFs.

RL MID-2 ANSWERS

UNIT- III

1. Explain the Markov property and its importance in MDPs.

PDF Reference: Pages 21–25 of UNIT-3.pdf.

1. Definition

The Markov Property is an important concept in Reinforcement Learning (RL) and Markov Decision Processes (MDPs). It states that the future depends only on the current state and action, not on the complete past history.

2. Mathematical Representation

$P(S_{t+1} \mid S_t, A_t)$

Where:

- S_t = Current state
- A_t = Current action
- S_{t+1} = Next state
- P = Transition probability

3. Diagram

Current State (S_t)

↓

Action (A_t)

↓

Environment Transition

↓

Next State (S_{t+1})

4. Real-life examples

A. Elevator example

As explained on page 23 of the PDF:

- States: Floor 1, Floor 2 and Floor 3.
- Actions: Move Up or Move Down.
- If the elevator is at Floor 2 and the action is Move Up, it goes to Floor 3.

The previous floor does not matter in this simplified example. The current floor and action determine the next state.

B. Chess game

According to page 25 of the PDF, the current chessboard position can be used as the state representation in the example. If the state contains all relevant information, the complete sequence of past moves need not be stored separately.

C. Weather prediction

The PDF also explains when the Markov Property may fail. Knowing only that the temperature is \(30^\\circ C\) may not be enough to predict the next temperature accurately. Additional information, such as time of day, humidity and season, may be required.

Therefore, the state must include enough relevant information to support accurate future predictions.

5. Importance of the Markov Property

1. Memory reduction: The agent need not store the complete past history.
2. Simple calculations: Future-state predictions depend on the current state and action.
3. Bellman equations: It supports value-function calculations.
4. Efficient computation: It simplifies planning and decision-making.
5. RL algorithms: It supports methods such as Dynamic Programming, Q-learning and Policy Iteration.

6. Limitation

If the current state does not contain enough relevant information, the Markov Property may fail. For example, temperature alone may not be sufficient to predict future weather accurately.

Conclusion

The Markov Property is fundamental to MDPs because it simplifies prediction, value calculations and decision-making by using the current state and action instead of the complete past history.

2. Explain the agent–environment interaction in a finite MDP with a diagram.

PDF Reference: UNIT-3.pdf, pages 1–13.

1. Definition

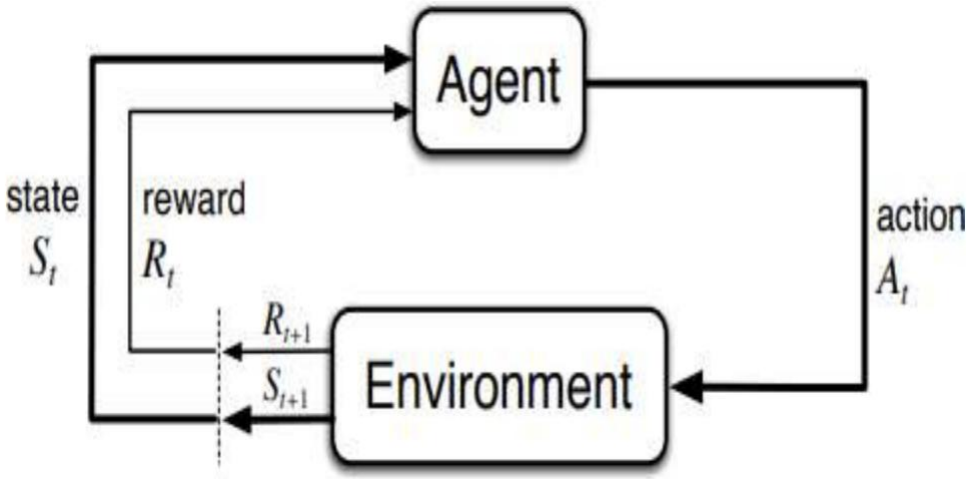
In Reinforcement Learning (RL), the Agent–Environment Interface describes how the agent and environment communicate with each other at every time step. The agent observes the current state, selects an action, and receives a reward and the next state from the environment.

A finite MDP has a finite number of states and actions. The goal of the agent is to maximize the total reward over time.

2. Main Components

1. Agent: The learner or decision-maker that observes the state, chooses actions, and learns from rewards.
2. Environment: Everything outside the agent. It receives actions, changes its state, and provides rewards and the next state.
3. State (S_t): The current situation of the agent.
4. Action (A_t): The choice made by the agent.
5. Reward (R_{t+1}): Numerical feedback provided by the environment after an action.
6. Next State (S_{t+1}): The new situation reached after the action.

3. Diagram of Agent–Environment Interaction



4. Working and Explanation

1. State: The environment provides the current state (S_t) to the agent.
2. Action: The agent selects an action (A_t) according to its policy.
3. Environment response: The environment responds to the selected action.
4. Reward: The environment provides a numerical reward (R_{t+1}).
5. Next state: The environment moves to a new state (S_{t+1}) and provides it to the agent.
6. Learning: The agent uses the reward and next state to improve its future decisions.
7. Repeat: This interaction continues until the task is completed or continues indefinitely, depending on the task.

5. Example: Robot Moving in a Grid World

Consider a robot moving towards a goal in a grid.

- State: Current position of the robot.
- Action: Move Right or Move Down.
- Reward: +10 for reaching the goal and −5 for a wrong move.
- Next state: The new grid position after the action.

Example interaction:

State S_1 → Action: Right → State S_2 → Action: Down → Goal

The robot learns to choose actions that maximize the total reward and reach the goal.

Conclusion

The Agent–Environment Interface is the basic interaction loop in a finite MDP. The agent selects actions, and the environment returns rewards and new states. Through repeated interaction, the agent learns to choose actions that maximize cumulative reward.

3. Derive the unified notation for episodic and continuing tasks.

PDF Reference: UNIT-3.pdf, pages 14–21.

1. Definition

In Reinforcement Learning (RL), the return (G_t) is the total future reward received by the agent from time step \(t\) onward. Unified notation uses one return formula for both episodic tasks, which have an ending, and continuing tasks, which do not have a terminal state.

The agent aims to maximize the return over time.

2. Return in Reinforcement Learning

The return is the sum of future rewards, taking the discount factor (γ) into account.

The unified return equation is:

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \ldots$

Where:

- G_t = Return at time step t
- R_{t+1} = Reward received at the next time step
- γ (Gamma) = Discount factor, where $0 \leq \gamma \leq 1$
- t = Current time step

3. Return for Episodic Tasks

An episodic task is a task that ends after a finite number of steps. It has a terminal state.

Examples:

- Playing a game level
- Reaching a destination
- Solving a maze

The return for an episodic task is:

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots + \gamma^{T-t-1} R_t$

Where T is the final time step.

Explanation:

- The agent receives rewards at each step.
- The rewards are discounted using γ.
- When the agent reaches the terminal state, the episode ends and further rewards stop.
- Therefore, the return is a finite sum.

4. Return for Continuing Tasks

A continuing task is a task that does not end and has no terminal state.

Examples:

- Temperature control system
- Robot cleaning continuously
- Stock-trading bot

The return for a continuing task is:

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$ (continuing to infinity)

Explanation:

- The agent continues interacting with the environment.
- Rewards continue to be received at future time steps.
- There is no terminal state.
- The discount factor γ must be less than 1 to keep the infinite sum finite under bounded rewards.

5. Derivation of Unified Notation

Step 1: General return equation

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

Step 2: Episodic tasks

In episodic tasks, the episode ends at time T. Rewards become zero after the terminal state, so the sum stops naturally.

$G_t = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-t-1} R_t$

Step 3: Continuing tasks

In continuing tasks, there is no terminal state, so rewards continue indefinitely.

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots (\infty)$

Here, $\gamma < 1$ keeps the infinite sum finite when rewards are bounded.

Step 4: Unified notation

Thus, the same equation represents both episodic and continuing tasks:

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

For episodic tasks, the sum stops at the terminal state. For continuing tasks, it extends indefinitely.

7. Advantages of Unified Notation

1. It provides a single formula for episodic and continuing tasks.
2. It simplifies the representation of future rewards.
3. It makes the treatment of both types of tasks consistent.

4. Explain the Bellman expectation equation for the state-value function with a Suitable example.

PDF Reference: UNIT-3.pdf, approximately pages 32–43.

1. Definition

The Bellman Expectation Equation expresses the value of a state as the expected immediate reward plus the discounted value of the next state, when the agent follows a policy π.

The state-value function, $V\pi(s)$, represents the expected return when the agent starts in state s and follows policy π.

2. Bellman Expectation Equation

The Bellman expectation equation for the state-value function is:

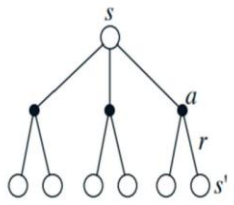
$$V\pi(s) = \Sigma_a \, \pi(a|s) \, \Sigma_{s',r} \, p(s', r \mid s, a) \, [r + \gamma V\pi(s')]$$

Where:

- $V\pi(s)$ = State-value function under policy π
- $\pi(a|s)$ = Probability of selecting action a in state s
- $p(s', r \mid s, a)$ = Probability of reaching next state s′ and receiving reward r
- r = Immediate reward
- γ = Discount factor
- $V\pi(s')$ = Value of the next state

The equation calculates the expected value by considering the immediate reward and the discounted value of the next state.

Backup Diagram for $V_\pi(s)$



3. Suitable Numerical Example

Consider the following states:

A → B → Goal

Given:

- Reward from A to B = 2
- Reward from B to Goal = 5
- Discount factor γ = 0.9
- Policy: Always move forward

Step 1: Calculate the value of Goal

The Goal is a terminal state, so:

$V\pi(Goal) = 0$

Step 2: Calculate the value of state B

$V\pi(B) = 5 + 0.9 \times 0$

$V\pi(B) = 5$

Step 3: Calculate the value of state A

$V\pi(A) = 2 + 0.9 \times V\pi(B)$

$V\pi(A) = 2 + 0.9 \times 5$

$V\pi(A) = 2 + 4.5$

$V\pi(A) = 6.5$

Final Values:

State	Value
Goal	0
B	5
A	6.5

4. Explanation

- The value of state B is 5 because it gives an immediate reward of 5 and the Goal has value 0.
- The value of state A is 6.5 because it provides an immediate reward of 2 and leads to state B, which has value 5.
- Thus, the state-value function considers both immediate and future rewards.

Conclusion

The Bellman expectation equation calculates the expected value of a state under a given policy by combining the immediate reward with the discounted value of the next state. It is important for evaluating states in a Markov Decision Process.

5. Derive the Bellman equation for the action-value function.

PDF Reference: UNIT-3.pdf, pages 32–43 (Value Functions section).

1. Definition

The action-value function, denoted by $Q\pi(s, a)$, represents the expected return when the agent starts in state s, takes action a, and then follows policy π.

It helps determine how good a particular action is in a given state.

2. Mathematical Representation

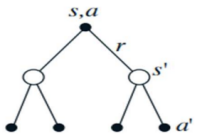
$Q\pi(s, a) = E\pi[G_t \mid S_t = s, A_t = a]$

Where:

- $Q\pi(s, a)$ = Action-value function under policy π
- G_t = Return from time step t
- S_t = Current state
- A_t = Current action
- $E\pi$ = Expected value when following policy π

Backup Diagram for ActionValue function

Numerical Example:



3. Derivation of the Bellman Equation

Step 1: Write the return equation

$G_t = R_{t+1} + \gamma G_{t+1}$

Step 2: Define the action-value function

$Q\pi(s, a) = E\pi[G_t \mid S_t = s, A_t = a]$

Substituting the return equation:

$Q\pi(s, a) = E\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$

Step 3: Consider the next state and reward

After taking action a in state s, the environment produces the next state s′ and reward r.

The agent then follows policy π in the next state. Therefore, the expected future action-value is:

$\Sigma_{a'} \pi(a' \mid s') Q\pi(s', a')$

Step 4: Obtain the Bellman equation

Thus, the Bellman expectation equation for the action-value function is:

$Q\pi(s, a) = \Sigma_{s',r} p(s', r \mid s, a) [r + \gamma \Sigma_{a'} \pi(a' \mid s') Q\pi(s', a')]$

This equation represents the expected immediate reward plus the discounted expected future action-value.

4. Suitable Numerical Example

Consider state A with two possible actions.

Immediate reward	Next-state value
1	2

2	5

Given: Discount factor $\gamma = 0.9$.

For action Left:

$Q(A, Left) = 1 + 0.9 \times 2$

$Q(A, Left) = 2.8$

For action Right:

$Q(A, Right) = 2 + 0.9 \times 5$

$Q(A, Right) = 6.5$

Therefore, action Right has the higher value in this example.

5. Diagram

State A → Select action → Receive immediate reward → Reach next state → Calculate future action-value

Conclusion

The Bellman equation for the action-value function expresses $Q\pi(s, a)$ in terms of the immediate reward and the expected discounted action-value of the next state. It helps evaluate actions and supports decision-making in Markov Decision Processes.

6. Explain optimal value functions and their significance.

PDF Reference: UNIT-3.pdf, pages 44–51.

1. Definition

An optimal value function gives the maximum expected return that an agent can obtain by following an optimal policy. It helps the agent choose actions that maximize the long-term reward.

There are two types of optimal value functions:

1. Optimal State-Value Function — V*(s)
2. Optimal Action-Value Function — Q*(s, a)

2. Optimal State-Value Function — V*(s)

The optimal state-value function represents the maximum expected return achievable from state s under an optimal policy.

Formula:

$V(s) = \max_a Q(s, a)$**

Where:

- V*(s) = Optimal value of state s
- Q*(s, a) = Optimal action-value for action a in state s
- $\max_a$ = Maximum over all available actions

3. Optimal Action-Value Function — Q*(s, a)

The optimal action-value function represents the maximum expected return obtained by taking action a in state s and then following an optimal policy.

Formula:

Q(s, a) = Expected return from taking action a in state s and then acting optimally.*

4. Suitable Numerical Example

Consider three states: S1 (Start), S2 (Middle), and S3 (Goal).

Discount factor: $\gamma = 0.9$.

The Goal is a terminal state, so V*(S3) = 0.

Step 1: Calculate V(S2)*

Immediate reward	Next state
4	S3
2	S3

For A1:

$Q^*(S2, A1) = 4 + 0.9 \times 0 = 4$

For A2:

$Q^*(S2, A2) = 2 + 0.9 \times 0 = 2$

Therefore:

V(S2) = max(4, 2) = 4*

Best action = A1.

Step 2: Calculate V(S1)*

Immediate reward	Next state
5	S2
1	S3

For A1:

$Q^*(S1, A1) = 5 + 0.9 \times 4 = 8.6$

For A2:

$Q^*(S1, A2) = 1 + 0.9 \times 0 = 1$

Therefore:

V(S1) = max(8.6, 1) = 8.6*

Best action = A1.

Final values:

State	Optimal value	Best action
S1	8.6	A1
S2	4	A1
S3	0	—

5. Diagram

S1 → A1 → S2 → A1 → S3 (Goal)

The agent chooses actions that provide the highest expected long-term reward.

6. Significance of Optimal Value Functions

1. Maximum reward: Finds the best expected return from each state.
2. Optimal policy: Helps determine the best action in each state.
3. Decision-making: Helps the agent select actions based on long-term rewards.
4. Value Iteration: Supports finding optimal state values.
5. Policy Iteration: Helps evaluate and improve policies.
6. Q-Learning: Uses action-value estimates to learn better actions.

Conclusion

Optimal value functions identify the maximum expected return achievable from a state or state-action pair. They are important in Reinforcement Learning because they help determine optimal policies and guide the agent toward actions that maximize long-term rewards.

UNIT- IV

1. Explain Monte Carlo prediction for estimating state-value functions.

PDF Reference: UNIT-4.pdf, Monte Carlo Prediction section, approximately pages 6–9.

UNIT-4.pdf

1. Definition

Monte Carlo Prediction is a Reinforcement Learning method used to estimate the state-value function, $V\pi(s)$, by averaging the returns obtained from multiple episodes while following a policy π.

It does not require a model of the environment. It uses the rewards collected from completed episodes to estimate the value of each state.

2. Mathematical Formula

Return:

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

State-value estimation:

$V\pi(s) = (G_1 + G_2 + \ldots + G_n) / N$

Where:

- $V\pi(s)$ = Estimated value of state s under policy π
- $G_1, G_2, \ldots, G_n$ = Returns observed from state s
- N = Number of visits to state s
- γ = Discount factor

3. Working of Monte Carlo Prediction

1. Generate episodes: The agent interacts with the environment by following policy π until each episode ends.
2. Calculate returns: Calculate the total discounted reward from each visited state until the end of the episode.
3. Record returns: Store the returns obtained whenever state s is visited.
4. Average returns: Calculate the average of the recorded returns.
5. Estimate state value: Use the average return as the estimated value $V\pi(s)$.

4. Example from the PDF

The PDF illustrates Monte Carlo Prediction using a grid-world example. The agent moves through states and receives rewards, and the returns from visited states are averaged to estimate their values.

The example reports the following estimated state values:

State	Estimated value
S1	7.5

S2	7
S3	2.5

These values represent the estimated returns for the states under the policy used in the example.

5. Types of Monte Carlo Prediction

1. **First-Visit Monte Carlo:** Updates the state value using the return following the first occurrence of that state in an episode.
2. **Every-Visit Monte Carlo:** Updates the state value using the return following every occurrence of that state in an episode.

Conclusion

Monte Carlo Prediction estimates state-value functions by averaging returns from completed episodes. It is useful when the environment model is unknown and helps evaluate states under a given policy.

2. Explain off-policy Monte Carlo control using importance sampling and discuss its advantages and limitations.

PDF Reference: UNIT-4.pdf, approximately pages 12–16 (Off-Policy Methods and Importance Sampling).

1. Definition

Off-policy Monte Carlo Control is a Reinforcement Learning method in which the agent learns or improves a target policy (π) while collecting experience using a different behavior policy (b).

Importance sampling adjusts the returns collected under the behavior policy so they can be used to evaluate the target policy.

2. Important Terms

1. Target policy (π): The policy that the agent wants to evaluate or improve.
2. Behavior policy (b): The policy used to generate episodes and collect experience.
3. Importance sampling: A technique used to correct the returns because the behavior policy differs from the target policy.
4. Return (G_t): The cumulative discounted reward from time step t.

3. Importance-Sampling Formula

The importance-sampling ratio for one action is:

$\rho_t = \pi(A_t \mid S_t) / b(A_t \mid S_t)$

For a complete episode from time t to T−1:

$\rho_{t:t+1} = \prod_{k=t}^{T-1} [\pi(A_k \mid S_k) / b(A_k \mid S_k)]$

The importance-sampled return is:

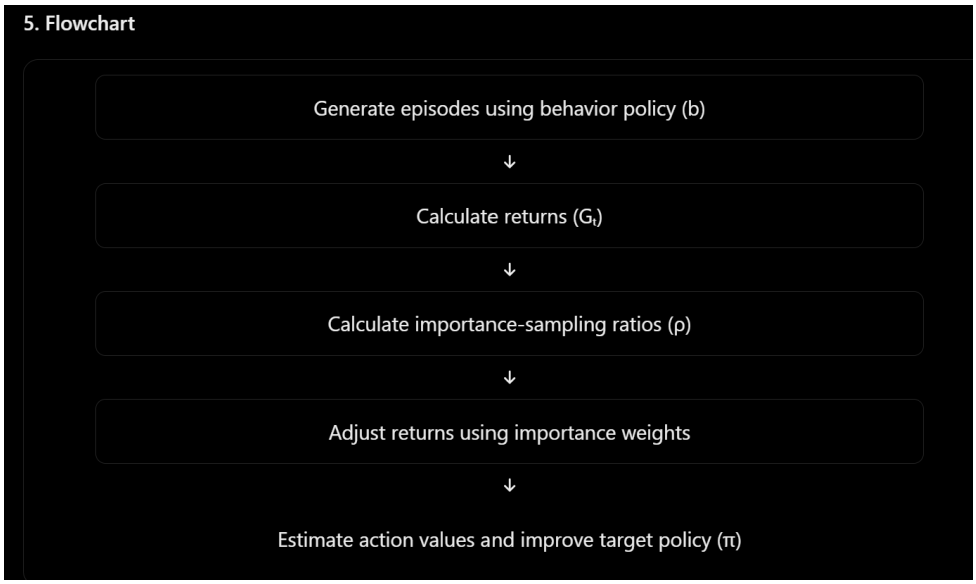
$G'_t = \rho_{t:t+1} \times G_t$

Where:

- $\pi(A_t \mid S_t)$ = Probability of selecting action A_t under the target policy.
- $b(A_t \mid S_t)$ = Probability of selecting action A_t under the behavior policy.
- ρ = Importance-sampling ratio.
- G_t = Original return.

4. Working of Off-Policy Monte Carlo Control

1. Generate episodes: Follow the behavior policy b to collect experience.
2. Calculate returns: Calculate the return G_t for the observed state-action pairs.
3. Calculate importance weights: Find the probability ratio between the target and behavior policies.
4. Adjust returns: Multiply the observed returns by the importance-sampling weights.
5. Estimate action values: Use the adjusted returns to estimate the target policy's action values.
6. Improve the policy: Update the target policy to choose actions with better estimated returns.
7. Repeat: Continue collecting episodes and improving the policy.



6. Types of Importance Sampling

Type	Characteristics
Ordinary importance sampling	Unbiased, but may have high variance
Weighted importance sampling	Lower variance, but slightly biased

7. Advantages

1. Policy flexibility: Learns about a target policy without directly following it.
2. Data reuse: Reuses previously collected experience from other policies.
3. Exploration: Allows the behavior policy to explore while the target policy is evaluated or improved.
4. Policy improvement: Supports learning better policies from sampled episodes.

8. Limitations

1. High variance: Estimates can have high variance when the target and behavior policies differ significantly.
2. Probability requirements: The action probabilities of both policies must be known.
3. Weight instability: Large importance-sampling ratios can make estimates unstable.

3. Describe the Monte Carlo Control with Exploring Starts algorithm with a suitable example.

PDF Reference: UNIT-4.pdf, approximately pages 2–4, Monte Carlo Methods and Exploring Starts section.

1. Definition

Monte Carlo Control with Exploring Starts is a Reinforcement Learning method used to find an optimal policy by estimating action values from complete episodes.

In this method, each episode starts from a randomly selected state and action, ensuring that every state-action pair has a chance to be explored.

2. Algorithm Steps

1. Initialize the action-value function Q(s, a) and policy π.
2. Start each episode from a randomly selected state-action pair.
3. Follow the current policy to generate the rest of the episode until it terminates.
4. Calculate the return G_t for each visited state-action pair.
5. Update Q(s, a) by averaging the returns observed for that pair.
6. Improve the policy by choosing the action with the highest estimated value.
7. Repeat the process until the policy approaches the optimal policy.

3. Important Formula

Return:

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$

Action-value update:

Q(s, a) = Average of the returns observed for state-action pair (s, a).

The policy is improved by selecting the action with the highest action value:

$\pi(s) = \arg\max_a Q(s, a)$

4. Suitable Example

Consider a robot moving from a starting state to a goal.

- States: S1, S2 and S3 (Goal).
- Actions: A1 = Move Right; A2 = Move Left.
- The robot receives rewards based on its actions and the state it reaches.

Suppose the robot explores the following state-action pairs and obtains these returns:

State	Action	Observed return	Estimated Q-value
S1	Move Right	8	8
S1	Move Left	3	3
S2	Move Right	5	5
S2	Move Left	1	1

Step 1: The robot begins episodes from randomly selected state-action pairs.

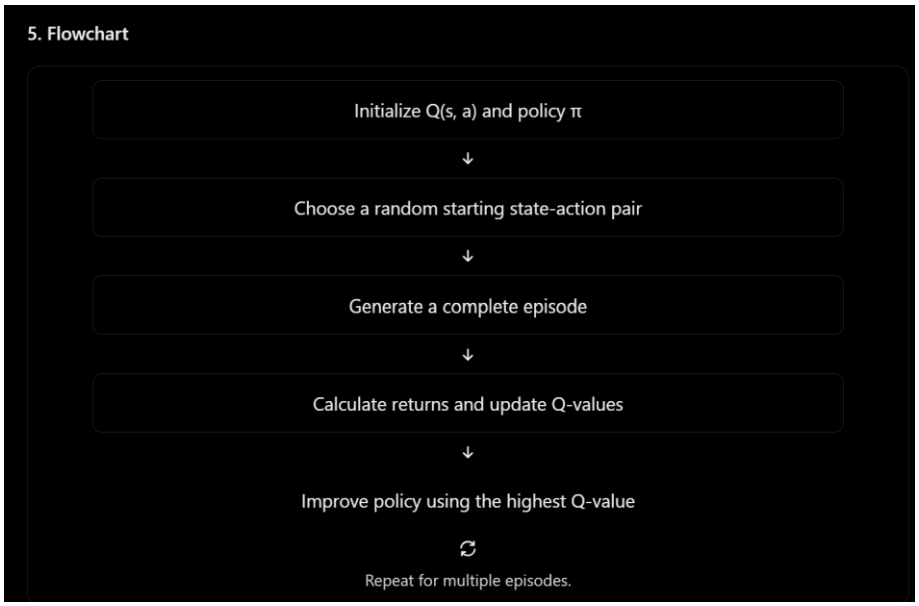
Step 2: It follows the policy until the episode ends and calculates the returns.

Step 3: It updates the Q-values using the observed returns.

Step 4: It improves the policy by selecting the action with the highest Q-value.

At S1, Move Right has a higher value (8) than Move Left (3). Therefore, the improved policy selects Move Right at S1.

Note: The numerical values above illustrate the algorithm; they are not presented as a specific numerical example from the PDF.



6. Advantages

1. Encourages exploration of different state-action pairs.
2. Estimates action values using actual returns from complete episodes.
3. Improves the policy through repeated evaluation and action selection.
4. Does not require a model of the environment.

7. Limitation

The method relies on the exploring-starts assumption: every state-action pair must have a chance of being selected as the starting pair. This may be difficult to guarantee in some environments.

4. Compare first-visit and every-visit Monte Carlo methods and explain their application in control problems.

Definition

First-Visit Monte Carlo: Estimates a state's value using the return following only its first occurrence in each episode.

Every-Visit Monte Carlo: Estimates a state's value using the returns following every occurrence of that state in an episode.

Comparison Between First-Visit and Every-Visit Monte Carlo Methods

Feature	First-Visit Monte Carlo	Every-Visit Monte Carlo
1. Meaning	Uses the first visit to a state.	Uses every visit to a state.
2. Value update	Updates once per state per episode.	Updates every time the state is visited.
3. Returns used	Uses the return after the first visit.	Uses returns after all visits.
4. Repeated visits	Ignores later visits in the same episode.	Includes repeated visits.
5. Experience used	Uses fewer returns per episode.	Uses more returns per episode.
6. Suitable for	Learning from first occurrences.	Learning when states recur frequently.
7. Value estimation	Averages first-visit returns across episodes.	Averages returns from all visits across episodes.
8. Updating frequency	Less frequent updates for each state within an episode.	More frequent updates for repeatedly visited states.
9. Main focus	First occurrence of a state.	Every occurrence of a state.
10. Control use	Helps estimate values for policy improvement.	Uses all observed visits to estimate values for policy improvement.

Application in Control Problems

1. Generate complete episodes using the agent's policy.
2. Calculate returns for visited state-action pairs.
3. Update action values using First-Visit or Every-Visit Monte Carlo.
4. Improve the policy by selecting actions with higher estimated values.
5. Repeat across episodes to learn a better policy.

Example: In robot navigation, the robot estimates returns for Move Left and Move Right and chooses actions with higher estimated values.

5. Explain incremental implementation of Monte Carlo methods.

1. Definition

Incremental implementation is a method of updating the value function step by step as new episodes or data arrive, instead of storing all previous returns and calculating their average again.

It is memory-efficient and supports continuous learning.

2. Need for Incremental Implementation

- Storing all past returns requires more memory.
- Recalculating averages using all previous data is expensive.
- Incremental updates allow the value estimate to be updated whenever new experience is available.

3. Incremental Update Formula

For the action-value function:

$Q(s, a) \leftarrow Q(s, a) + \alpha [G_t - Q(s, a)]$

Where:

- $Q(s, a)$ = Current action-value estimate
- G_t = Newly observed return
- α = Learning rate
- $G_t - Q(s, a)$ = Difference between the observed return and current estimate

The same incremental principle can be used to update a state-value estimate V(s).

4. Working with an Example

Suppose the current action-value estimate is:

$Q(s, a) = 4$

New return: $G_t = 8$

Learning rate: $\alpha = 0.5$

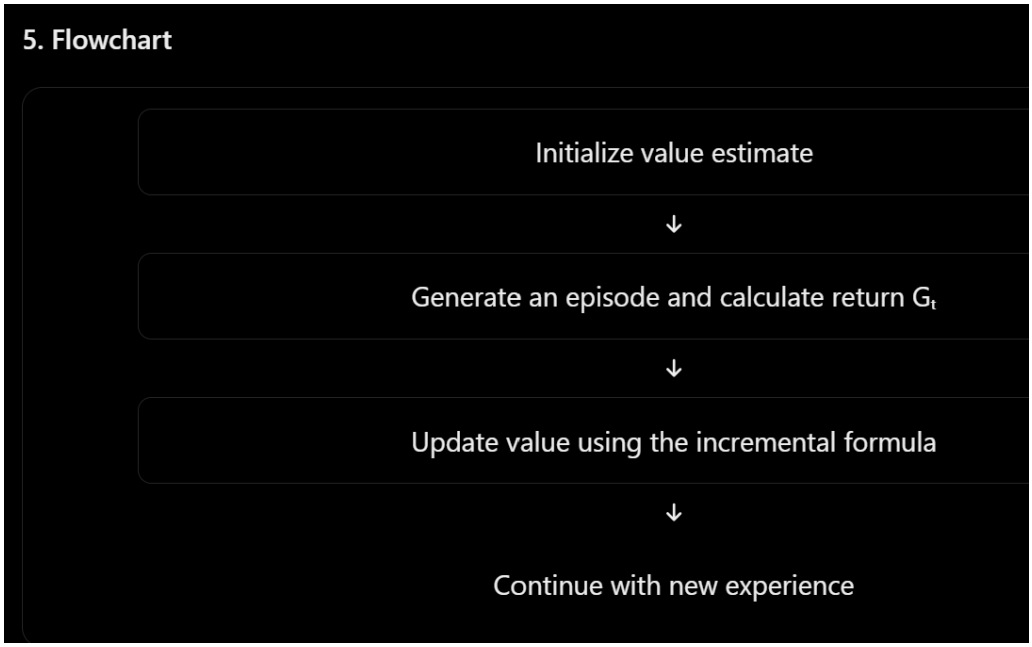
Using the formula:

$Q(s, a) \leftarrow 4 + 0.5(8 - 4)$

$Q(s, a) \leftarrow 4 + 0.5 \times 4$

$Q(s, a) = 6$

Therefore, the action-value estimate is updated from 4 to 6 using the new return.



6. Advantages

1. Requires less memory because all previous returns need not be stored.
2. Avoids recalculating the average from the complete history.
3. Supports online or continuous learning.
4. Updates estimates as new experience becomes available.

Conclusion

Incremental implementation updates Monte Carlo value estimates step by step using new returns. It reduces memory requirements and computational effort while allowing the agent to learn continuously from experience.

UNIT- V

1.Describe Samuel's Checkers Player and its learning mechanism.

PDF pages used: UNIT-5.pdf, pages 2–4; unit-5-TEXTBOOK.pdf, printed pages 279–282.

1. Definition

Samuel's Checkers Player was an early artificial intelligence program developed by Arthur Samuel in the 1950s. It was designed to play checkers and improve its performance by learning from experience.

It combined heuristic search methods with learning techniques. Its two main learning mechanisms were rote learning and learning by generalization.

2. Main components

1. **Minimax Search:** Explores possible moves and selects the best move while considering the opponent's response.
2. **Alpha-Beta Pruning:** Eliminates unnecessary search branches to improve efficiency.
3. **Evaluation Function:** Assigns scores to board positions based on features such as piece count and positioning.

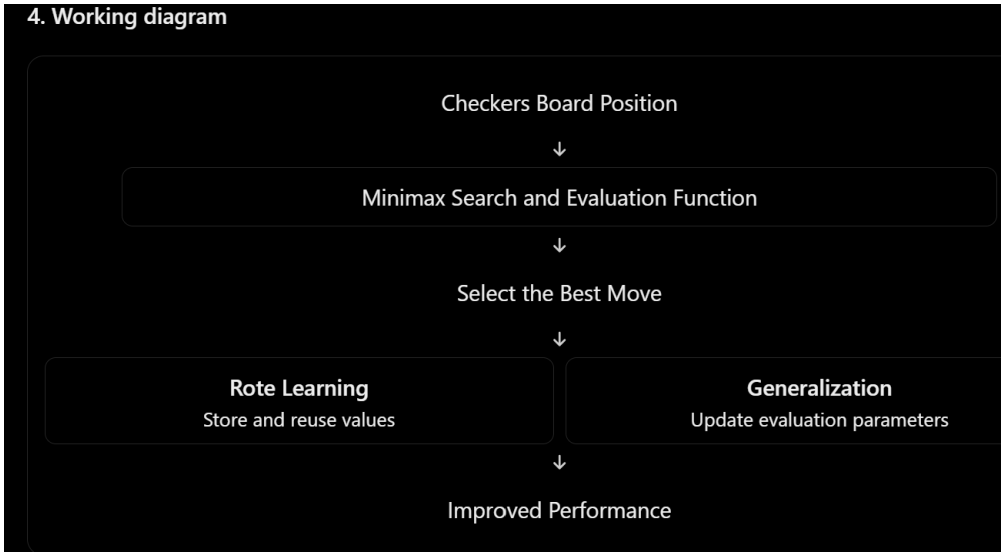
3. Learning mechanisms

A. Rote Learning

- Stores board positions with their evaluated values.
- Reuses stored values when the same positions occur again.
- Reduces repeated computation.

B. Learning by Generalization

- Adjusts evaluation-function parameters based on game experience.
- Learns by playing games against another version of itself.
- Improves position evaluation and future move selection.



5. Advantages

1. Improves performance through experience.

2. Rote learning reduces repeated computation.
3. Alpha-beta pruning improves search efficiency.
4. Generalization helps evaluate previously unseen positions.

6. Limitations

1. Searching possible moves requires computational resources.
2. Rote learning depends on previously stored positions and values.
3. Generalization learning can sometimes produce poor evaluation functions and reduce performance.

7. Achievement and conclusion

According to the unit notes, Samuel's program defeated checkers player Robert Nealey in 1962. It demonstrated that a computer could learn and improve without being explicitly reprogrammed for every situation.

Conclusion: Samuel's Checkers Player combined minimax search, rote learning, and learning by generalization to improve its playing ability through experience.

2.Discuss the application of Reinforcement Learning in elevator dispatching.

Page numbers: UNIT-5.pdf, Pages 7–9; Textbook, Pages 285–289.

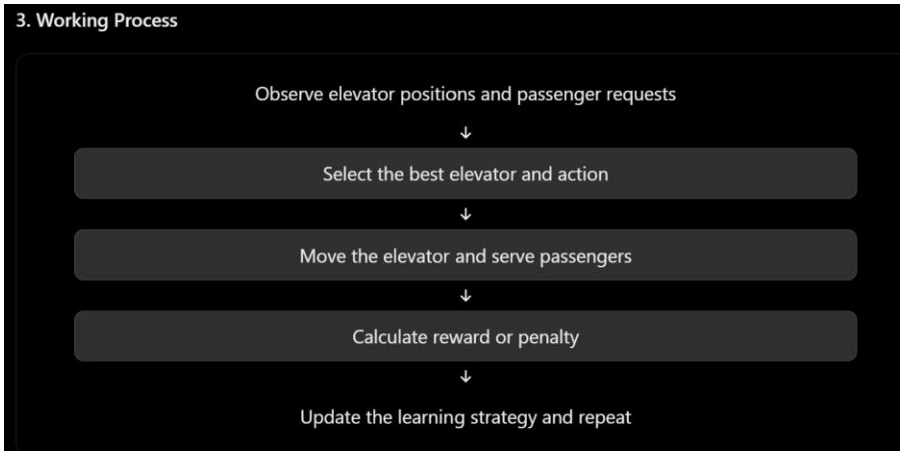
1. Definition

Reinforcement Learning (RL) in elevator dispatching is a technique in which an elevator system learns from experience to assign elevators to passenger requests efficiently.

2. Working Mechanism

The main components are:

1. State: Elevator positions, directions, passenger loads and pending requests.
2. Action: Assign an elevator to a request or change its direction.
3. Reward: Positive rewards for reducing waiting time and penalties for excessive waiting or energy usage.
4. Policy: The strategy learned by the system to select suitable actions.



4. Techniques Used

- Q-Learning: Learns the value of actions in different states.
- Deep Reinforcement Learning: Uses deep neural networks to handle complex elevator systems.
- Multi-Agent RL: Allows multiple elevators to coordinate their actions.

5. Advantages

- Reduces passenger waiting and travel time.
- Adapts to changing passenger traffic.
- Improves elevator workload distribution.
- Helps reduce unnecessary energy consumption.

6. Limitations

- Requires sufficient training experience and computational resources.
- May face difficulties in rare or unexpected situations.

Conclusion: Reinforcement Learning helps elevator systems learn efficient dispatching strategies by interacting with the environment and improving through rewards and penalties.

3.Analyse how RL can be applied to solve Dynamic Channel Allocation problem based on network conditions.

PDF page references:

- unit-5-TEXTBOOK.pdf — PDF pages 18–22; printed textbook pages 291–295.
- UNIT-5.pdf — Dynamic Channel Allocation is not covered as a separate topic in these notes.

1. Definition

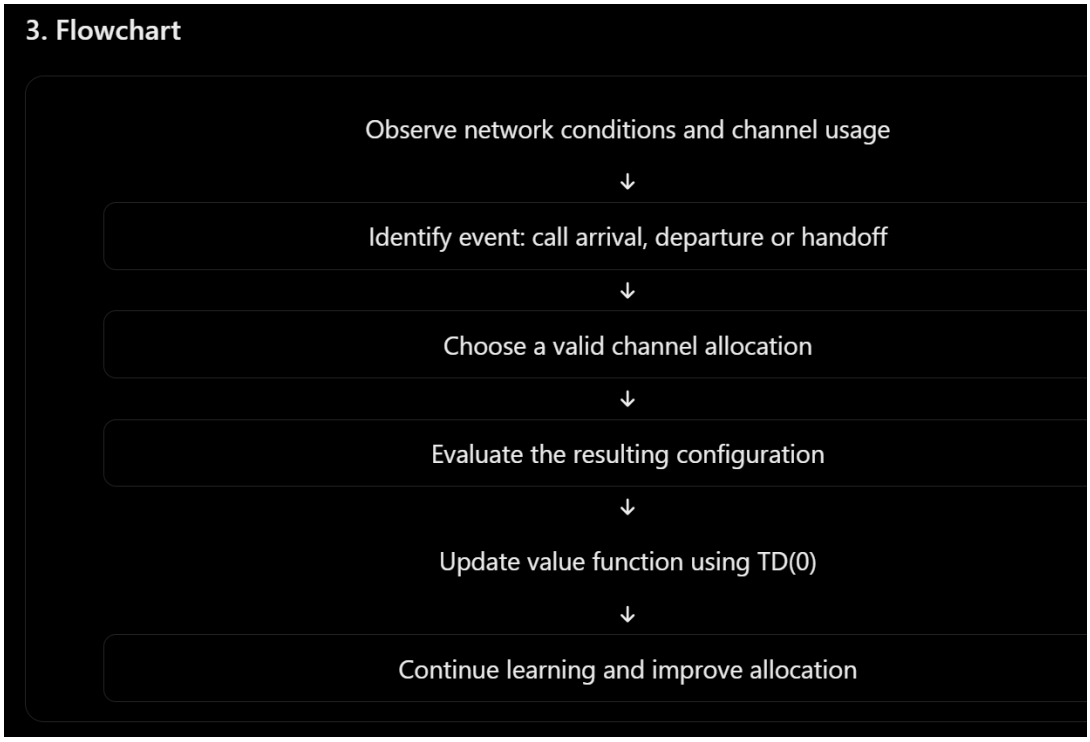
Dynamic Channel Allocation (DCA) is the process of assigning available communication channels to mobile calls in a cellular telephone network according to demand, while avoiding interference between nearby cells.

Reinforcement Learning (RL) helps the network learn better channel-allocation decisions from experience to minimize the number of blocked calls and use available bandwidth efficiently.

2. Working of Reinforcement Learning in DCA

1. Observe network conditions: Identify channel usage in each cell and whether a call arrives, ends, or is handed off from another cell.
2. Select an action: When a call arrives, assign an available channel that satisfies the channel-reuse constraint. If no suitable channel is available, block the call.
3. Evaluate the configuration: Estimate the value of the resulting channel arrangement using features such as channel availability and channel usage in nearby cells.
4. Learn from experience: The system updates its value-function weights using semi-Markov linear TD(0) learning.

5. Improve allocation: When a call ends and releases a channel, the system can reassign channels to create a better configuration.



4. Advantages

- Reduces the probability of calls being blocked.
- Adapts channel allocation to changing call-arrival rates across cells.
- Learns effective channel-use patterns through experience.
- Can make allocation decisions online with less computation than some complex optimization methods.

5. Conclusion

Reinforcement Learning provides an effective method for Dynamic Channel Allocation by learning from changing network conditions, maintaining channel-reuse constraints, and improving channel utilization to reduce blocked calls. The textbook reports that the RL method blocked calls less frequently than the compared fixed-assignment and borrowing-based methods in the studied simulations.

4. Describe job-shop scheduling and the role of RL in optimizing schedules.

Page numbers taken from:

- UNIT-5.pdf: Pages 10–11.
- unit-5-TEXTBOOK.pdf: Printed textbook pages 295–301 (PDF pages approximately 23–29).

1. Definition

Job-Shop Scheduling is an optimization problem in which a set of jobs must be completed using available resources or machines while satisfying time and resource constraints. The objective is to find a conflict-free schedule with the shortest possible total duration.

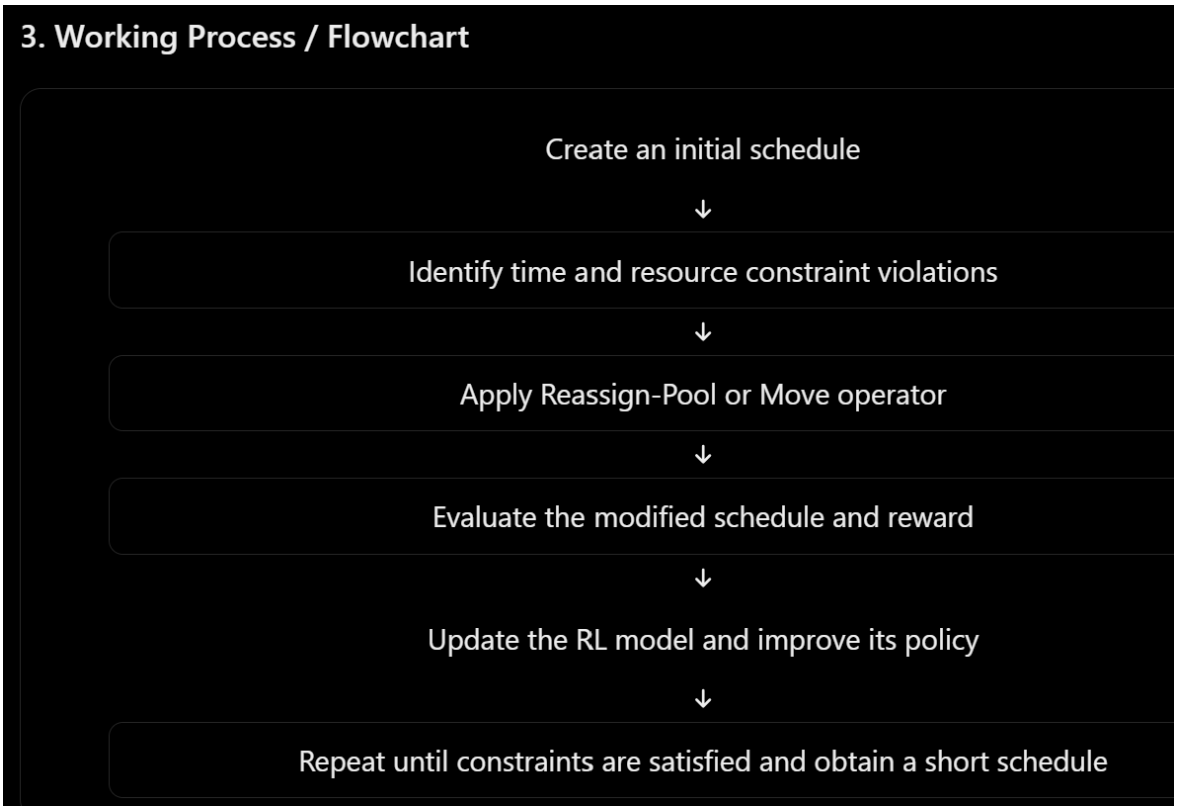
Reinforcement Learning (RL) helps learn a policy that can find good schedules efficiently for different scheduling problems.

2. Role of Reinforcement Learning

RL is applied to job-shop scheduling as follows:

1. State: The current schedule, including task timings and resource assignments.
2. Actions: Modify the schedule using two operators:
    - Reassign-Pool: Changes the resource pool assigned to a task.
    - Move: Moves a task to an earlier or later time when its resource requirements can be satisfied.
3. Reward: A small negative reward (−0.001) is given for each step that still has constraint violations. A final reward based on the Resource Dilation Factor (RDF) encourages shorter schedules.
4. Learning: The system uses TD(λ) learning and a neural network to estimate schedule values and improve its decisions.
5. Optimization: It learns to find conflict-free schedules with shorter duration and fewer repair steps.

3. Working Process / Flowchart



4. Advantages

- Learns to find good schedules with fewer repair steps.
- Helps reduce overall schedule duration.
- Reduces the need to design specialized scheduling rules manually.
- Learns a policy that can be applied to a class of related scheduling problems.

Example Application

The textbook describes **NASA's Space Shuttle** Payload Processing Problem (SSPP). Reinforcement Learning learns to schedule shuttle payload installation and testing tasks while satisfying time and resource constraints, aiming to find shorter schedules efficiently.

Limitations

- RL requires training and computational effort.
- Its effectiveness for complex scheduling problems is not conclusively established.
- Learning performance depends on the scheduling problem and the methods used.

---- THE END ----

BHAAI PDFS